28-10-2026

Review on 4626

group by new IDs by the file they're new in

pontoon/sync/core/entities.py

```
@@ -365,22 +365,20 @@ def restore_obsolete_entities(
     new_keys = next_entities.keys() - prev_entities.keys()
     if not new_keys:
         return {}
-    resource_ids = {next_entities[key].resource_id for key in new_keys}
-    entity_keys = {list(k) for k in {entity_key for _, entity_key in new_keys}}
+    keys_by_resource: dict[int, list[list[str]]] = defaultdict(list)
+    for key in new_keys:
+        keys_by_resource[next_entities[key].resource_id].append(list(key[1]))
```

turn ID into list from needed by DO array column

(file path, ID) pair

file's database ID

check on OR conditions

```
371 +    new_keys_q = Q()
372 +    for resource_id, entity_keys in keys_by_resource.items():
373 +        new_keys_q |= Q(resource_id=resource_id, key__in=entity_keys)
370 374  restored: dict[tuple[str, L10nId], Entity] = {}
371 375  for ent in (
372      -      Entity.objects.filter(
```

- address logic in comment 2 for the initial concern

This matches every key against every resource, so a key that is new only in `a.ftl` is also looked up in the other changed files. A large sync fetches a lot of rows to throw away. Could we OR one `Q(resource_id=…, key__in=…)` per resource instead?

```
375      -          .order_by("-pk")
376      -          .iterator()
     376 +      Entity.objects.filter(new_keys_q, obsolete=True).order_by("-pk").iterator()
377  377      ):
378  378          key = (changed_resources[ent.resource_id].path, tuple(ent.key))
379      -        next_ent = next_entities.get(key, None)
     379 +        next_ent = next_entities[key]
380  380          if (
381      -            next_ent is not None
382      -            and key not in prev_entities
383      -            and key not in restored
     381 +            key not in restored
```

not true by definition

Editor: Autocomplete

- add IMT leveraged work

```
84 +  }
85 +
86 +  toDOM() {
87 +    const span = document.createElement('span');
88 +    span.className = 'ghost-text';
89 +    span.setAttribute('aria-hidden', 'true');
90 +    span.textContent = this.text;
91 +    return span;
92 +  }
93 + }
94 +
95 + function ghostTextDecorations(view: EditorView): DecorationSet {
96 +   const ghost = getGhostText(view);
```

create element from user's code place it into DOM to make a reference